

run_oc_simple_time_sim_arbitrary(24).m
檔案
my_model(25).m
檔案
my_params(34).m
檔案
978-94-017-8533-4(5).pdf
PDF
EOM Derivation Check(2).pdf
PDF
SVDerrata(5).pdf
PDF

I want to use these simulation to study the tyre load and tyre force when car steers, the vehicle model comes from those book, document and discussion pdfs. For lateral load transfer, I think veh.d makes a big difference. What do you think of veh.d's effects on load transfer and vehicle dynamics? Moreover, do you think would it be possible to back-identify veh.d using the simulation data from the current code? If so, what kind of simulations should be done to collect the data and what kind of identification algorithm should be used in order to back-identify veh.d?

已思考 2m 50s >

Yes — in your current model, `veh.d` can make a large visible difference, especially in tyre load histories and steering transients. But the most important point is:

`veh.d` does not simply “increase total lateral load transfer.” It changes the path and timing of load transfer: direct/no-roll-center transfer versus elastic/damped roll transfer.

In your current parameters, `veh.d` = 0.7*`h`, `H` = `h` - `d`, and `q1` = `q2` = `d`, so with `h` = 0.453 m, you are using roughly `d` = 0.317 m and `H` = 0.136 m. That means your model assigns a large part of lateral load transfer to the direct/no-roll-center path. `my_params`

1. What `veh.d` does in your current equations

Your corrected Chapter 9 implementation uses

$$H = h - d$$

in the roll/pitch/yaw dynamics:

$$\theta = \frac{M - HX}{I} \begin{bmatrix} x \\ y \end{bmatrix} \& - \frac{J_z}{I} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \& \frac{1}{I} \begin{bmatrix} \ddot{x} \\ \ddot{y} \end{bmatrix}$$

$$\phi = \begin{bmatrix} \phi_1 \\ \phi_2 \end{bmatrix} = \begin{bmatrix} L_M + H Y \backslash N - H X \phi \end{bmatrix}$$

and

$$\dot{u} = a_x + w - H \dot{\theta} \quad \dot{v} = a_y - u + H \dot{\phi}$$

So, when `veh.d` increases, `H` = `h`-`d` decreases. That means the lateral force has a smaller roll moment arm about the VIP, so the body tends to roll less and the roll transient becomes less dominant. Your implementation document states the same compact Chapter 9 structure. `README_CH9_TOTPE_MODIFICATIONS`

But at the same time, your tyre-load equations use

$$\Delta Z_1 = \frac{Y_1 q_1}{t_1} + k_{\phi 1} \phi + c_{\phi 1} \dot{\phi},$$

$$\Delta Z_2 = \frac{Y_2 q_2}{t_2} + k_{\phi 2} \phi + c_{\phi 2} \dot{\phi},$$

and in your current parameters you set

$$q_1 = q_2 = d.$$

Therefore increasing `veh.d` directly increases the geometric/direct lateral load-transfer terms $Y_1 d/t_1$ and $Y_2 d/t_2$. The book's Chapter 9 formula also says the lateral load transfers become $\Delta Z_1 t_1 = Y_1 q_1 + k_{\phi 1} \phi + c_{\phi 1} \dot{\phi}$ and $\Delta Z_2 t_2 = Y_2 q_2 + k_{\phi 2} \phi + c_{\phi 2} \dot{\phi}$, and that with roll/pitch motion, load transfer depends explicitly on body angular accelerations. `978-94-017-8533-4`

So the two main effects oppose each other:

$$\text{larger } d \Rightarrow \left\{ \begin{array}{l} \text{larger direct/geometric load transfer,} \\ \text{smaller roll moment arm } H, \\ \text{smaller roll angle and roll-rate contribution,} \\ \text{faster/more immediate tyre-load redistribution.} \end{array} \right.$$

2. Why this changes tyre force and handling

Your tyre model uses each wheel's vertical load F_z directly in the force calculation. The tyre force expressions multiply by F_z , and the peak friction coefficients are also functions of F_z . So changing `veh.d` changes $F_{z,fl}$, $F_{z,fr}$, $F_{z,rl}$, $F_{z,rr}$, which then changes F_x , F_y , yaw moment, roll response, and the subsequent trajectory. `my_model`

The practical consequence is:

Larger `veh.d` usually makes the outside-wheel load rise more immediately during steering input. The body roll part becomes less important, so the tyre-load response has less suspension/roll lag.

Smaller `veh.d` usually makes load transfer pass more through roll stiffness/damping. Then the load transfer is more delayed, more frequency-dependent, and more affected by $k_{\phi 1}$, $k_{\phi 2}$, $c_{\phi 1}$, $c_{\phi 2}$.

This is why `veh.d` can strongly affect transient steering behaviour even if the steady-state total lateral load transfer is not changed dramatically.

A useful intuition is:

$$\Delta Z_1 t_1 + \Delta Z_2 t_2 = Y d + k_{\phi} \phi + c_{\phi} \dot{\phi}$$

when $q_1 = q_2 = d$. In quasi-steady cornering, the total left-right transfer is still mostly governed by $m a_y h$. But d changes how much is direct versus how much is suspension-roll-mediated. That changes front/rear tyre load distribution, transient tyre saturation, and therefore yaw response.

3. Is it possible to back-identify `veh.d` from current simulation data?

Yes, but with an important qualification:

From simulation data generated by the same model, yes, `veh.d` is identifiable if you log wheel loads, tyre forces, roll angle, and roll rate.

But from vehicle-motion data only — for example u , v , r , ϕ , δ without wheel loads or tyre forces — `veh.d` will be much harder to identify because it is coupled with tyre load sensitivity, roll stiffness, roll damping, CG height, and front/rear force distribution.

Your current postprocessing already stores the key quantities: `sim.z_phys`, `sim.acc`, `sim.load`, `sim.tyre`, `sim.u_phys`, and `sim.x_phys`. The code evaluates `model.f_load`, `model.f_tyre`, and `model.f_acc` at every output time, so the data needed for identification is already mostly available. `run_oc_simple_time_sim_arbitrary`

4. Best simple identification method: linear least squares

For your current model, because you set

$$q_1 = q_2 = d,$$

you can estimate d almost directly from the load-transfer equations.

From simulation data, compute

$$\Delta Z_1^{obs} = \frac{F_{z,fr} - F_{z,fl}}{2},$$

$$\Delta Z_2^{obs} = \frac{F_{z,rr} - F_{z,rl}}{2}.$$

Then use

$$t_1 \Delta Z_1^{obs} - k_{\phi 1} \phi - c_{\phi 1} \dot{\phi} = Y_1 d,$$

$$t_2 \Delta Z_2^{obs} - k_{\phi 2} \phi - c_{\phi 2} \dot{\phi} = Y_2 d.$$

Stack all time samples and both axes:

$$b_i = A_i d.$$

Then

$$\hat{d} = (A^T W A)^{-1} A^T W b.$$

This is the cleanest first estimator.

For each time sample:

$$b_1 = t_1 \Delta Z_1^{obs} - k_{\phi 1} \phi - c_{\phi 1} \dot{\phi},$$

$$b_2 = t_2 \Delta Z_2^{obs} - k_{\phi 2} \phi - c_{\phi 2} \dot{\phi},$$

$$A_1 = Y_1, \quad A_2 = Y_2.$$

So with many samples:

$$\hat{d} = \frac{\sum_i (Y_{1,i} b_{1,i} + Y_{2,i} b_{2,i})}{\sum_i (Y_{1,i}^2 + Y_{2,i}^2)}.$$

This estimator is simple, fast, and very useful for checking whether the simulation data truly contains enough information to recover `veh.d`.

Even better, I would first estimate q_1 and q_2 separately:

$$\hat{q}_1 = \frac{\sum_i Y_{1,i} \left(t_1 \Delta Z_{1,i} - k_{\phi 1} \phi_i - c_{\phi 1} \dot{\phi}_i \right)}{\sum_i Y_{1,i}^2},$$

$$\hat{q}_2 = \frac{\sum_i Y_{2,i} \left(t_2 \Delta Z_{2,i} - k_{\phi 2} \phi_i - c_{\phi 2} \dot{\phi}_i \right)}{\sum_i Y_{2,i}^2}.$$

Then check whether $\hat{q}_1 \approx \hat{q}_2$. If not, the single- d assumption is too restrictive, and your model should probably use front and rear no-roll-center heights separately.

5. What simulations should be done?

Use several steering experiments, not just one. I would use:

First: constant-speed lateral-only tests. Accelerate to target speed, release torque, wait for the no-torque/no-steering delay, then steer. This isolates lateral load transfer from longitudinal load transfer.

Use smooth steering inputs, not discontinuous steps. For ode15i especially, use your smoothed step steering because the non-smooth command can create DAE consistency trouble.

Recommended data set:

1. Smooth step steer at 40, 60, 80 km/h, with 2°, 5°, and 8° steering amplitudes.
This is good for transient roll/load-transfer timing.
2. Sine steer at fixed speeds, with frequencies such as 0.3, 0.6, 1.0, 1.5, and 2.0 Hz.
This is very useful because d affects the balance between immediate geometric transfer and delayed roll-mediated transfer.
3. Slow ramp steer / quasi-steady cornering.
This is useful for checking steady-state consistency, but it is not enough alone because steady-state total lateral transfer is mostly governed by h , not uniquely by d .
4. Left and right steering.
This cancels sign bias and helps detect asymmetry or numerical bias.
5. Avoid extreme tyre saturation at first.
Start with low-to-medium lateral acceleration. After the estimator works, repeat with higher lateral acceleration to see whether the identified d remains stable.

6. When linear LS is not enough

If you only have states and controls, not F_z and tyre forces, use nonlinear grey-box identification:

$$\min_d \sum_k \left\| y_{obs}(t_k) - y_{sim}(t_k; d) \right\|_W^2$$

where y could include F_x , a_y , r , ϕ , $\dot{\phi}$, and maybe F_y if available.

Use MATLAB `lsqnonlin` with bounds such as

$$0 \leq d \leq h.$$

But do not estimate too many parameters at once. If you try to identify

$$d, k_{\phi 1}, k_{\phi 2}, c_{\phi 1}, c_{\phi 2}, \text{ tyre parameters}$$

all together, the problem will become highly correlated and possibly non-identifiable. Start with only d ; then try $[q_1, q_2]$; only after that include stiffness/damping uncertainty.

7. My recommendation for your current project

For your current simulation study, I would do this in stages:

1. Run a parameter sweep first: $d/h = 0.1, 0.3, 0.5, 0.7, 0.9$.
Compare F_z , ΔZ_1 , ΔZ_2 , ϕ , $\dot{\phi}$, F_y , yaw rate, and lateral acceleration.
2. Use the linear least-squares estimator on simulation-generated data.
Verify that it recovers the original $\text{veh.d} = 0.7 \cdot h$.
3. Add artificial noise to F_z , Y_1 , Y_2 , and ϕ .
Check how robust the estimate is.
4. Try estimating q_1, q_2 separately.
This will tell you whether the single veh.d assumption is hiding front/rear differences.
5. Only then try nonlinear output-error identification.

The most important caution is: veh.d in your code is an effective no-roll-center/VIP height, not automatically a directly measurable suspension hardpoint parameter. If you identify it from simulation or real data, interpret it as an effective load-transfer parameter unless your suspension geometry model explicitly defines q_1, q_2 from kinematics.

📄 🗨 🔄 📄 ⋮ 📄 資料來源

Thank you for your advice, due to that I in soon future I really need to identify veh.d, but due to I am not sure whether it is really possible, I decide to start from using the cleanest simulation data coming from current simulations. Because even if clean simulation data cannot back-identify veh.d, in reality from much more noisier data it would be almost impossible to identify veh.d. So please double check your above answers, we need to make sure the suggested identifiability and identification methods are both rigorous and implementable.

已思考 1m 10s >

I double-checked the reasoning against your current code. My corrected conclusion is:

Yes, veh.d is identifiable from clean simulation data generated by the current code, but only under a very specific meaning of “identify”: you can identify the effective no-roll-center/VIP load-transfer height used in the Chapter 9 tyre-load equations.

This is implementable and rigorous if you log or reconstruct wheel loads, tyre forces, steering angle, roll angle, and roll rate. It is not guaranteed if you only use vehicle trajectory states like u, v, r, n, ξ , because then veh.d is strongly coupled with tyre parameters, roll stiffness, roll damping, and CG height.

1. The key correction to my previous answer

The clean linear least-squares estimator is rigorous for the load-transfer part of veh.d .

Your parameters define

$$d = 0.7h, \quad H = h - d, \quad q_1 = q_2 = d.$$

This is explicit in `my_params.m`: $\text{veh.d} = \text{veh.h} \cdot 0.7$, $\text{veh.H} = \text{veh.h} - \text{veh.d}$, and both veh.q1 and veh.q2 are set equal to veh.d .
`my_params`

In `my_model.m`, however, d enters the model through two different paths:

$$H = h - d$$

enters the roll/pitch/yaw/body dynamics, including $\backslash\text{theta} \backslash\text{phi} \backslash r \backslash \dot{u}$, and $\dot{v}$.
`my_model`

Meanwhile,

$$q_1 = q_2 = d$$

enters the lateral tyre-load equations:

$$\begin{aligned} \Delta Z_1 &= \frac{Y_1 q_1 + k_{\phi 1} \phi + c_{\phi 1} \dot{\phi}}{t_1}, \\ \Delta Z_2 &= \frac{Y_2 q_2 + k_{\phi 2} \phi + c_{\phi 2} \dot{\phi}}{t_2}. \end{aligned}$$

Your code implements exactly these equations before constructing the four tyre vertical loads.
`my_model`

So the previous linear estimator is not a complete dynamic output-error estimator of all effects of veh.d ; it is a direct estimator of q_1, q_2 . Since your current model sets $q_1 = q_2 = d$, that direct estimator also estimates veh.d .

That distinction matters.

2. Structural identifiability from clean simulation data

For each time sample, your code gives

$$\begin{aligned} F_{z,fl} &= \frac{1}{2} Z_1 - \Delta Z_1, & F_{z,fr} &= \frac{1}{2} Z_1 + \Delta Z_1, \\ F_{z,rl} &= \frac{1}{2} Z_2 - \Delta Z_2, & F_{z,rr} &= \frac{1}{2} Z_2 + \Delta Z_2. \end{aligned}$$

Therefore,

$$\Delta Z_1 = \frac{F_{z,fr} - F_{z,fl}}{2}, \quad \Delta Z_2 = \frac{F_{z,rr} - F_{z,rl}}{2}.$$

Then from your lateral load-transfer equations,

$$\begin{aligned} t_1 \Delta Z_1 - k_{\phi 1} \phi - c_{\phi 1} \dot{\phi} &= Y_1 q_1, \\ t_2 \Delta Z_2 - k_{\phi 2} \phi - c_{\phi 2} \dot{\phi} &= Y_2 q_2. \end{aligned}$$

So, if the data are clean and the model residual $h_{load} \approx 0$, q_1 and q_2 are linearly identifiable as long as the lateral axle forces are not zero over the whole dataset.

For separate front/rear identification:

$$\begin{aligned} \hat{q}_1 &= \frac{\sum_k Y_{1,k} \left(t_1 \Delta Z_{1,k} - k_{\phi 1} \phi_k - c_{\phi 1} \dot{\phi}_k \right)}{\sum_k Y_{1,k}^2}, \\ \hat{q}_2 &= \frac{\sum_k Y_{2,k} \left(t_2 \Delta Z_{2,k} - k_{\phi 2} \phi_k - c_{\phi 2} \dot{\phi}_k \right)}{\sum_k Y_{2,k}^2}. \end{aligned}$$

For your current single-parameter assumption $q_1 = q_2 = d$:

$$\hat{d} = \frac{\sum_k [Y_{1,k} b_{1,k} + Y_{2,k} b_{2,k}]}{\sum_k [Y_{1,k}^2 + Y_{2,k}^2]},$$

where

$$\begin{aligned} b_{1,k} &= t_1 \Delta Z_{1,k} - k_{\phi 1} \phi_k - c_{\phi 1} \dot{\phi}_k, \\ b_{2,k} &= t_2 \Delta Z_{2,k} - k_{\phi 2} \phi_k - c_{\phi 2} \dot{\phi}_k. \end{aligned}$$

This is not just a heuristic. It is the exact least-squares solution of the equation currently enforced by your model.

3. What data are needed from your current simulation?

The good news is that your current code already stores almost everything needed.

`sim.z_phys` stores the four solved tyre vertical loads, `sim.x_phys` stores states including ϕ and $\dot{\phi}$, `sim.u_phys` stores actual controls, `sim.acc`, `sim.load`, and `sim.tyre` are also saved during postprocessing.
`run_oc_single_time_sim_arbitrary`

Your `y_tyre` output is ordered as

$$[f_x f_l, f_x f_r, f_x r_l, f_x r_r, f_y f_l, f_y f_r, f_y r_l, f_y r_r, f_z f_l, f_z f_r, f_z r_l, f_z r_r].$$

Your `y_load` output includes h_{load} , the Chapter 9 reconstructed tyre loads, ΔZ , ΔZ_1 , and ΔZ_2 .
`my_model`

So you can reconstruct:

$$\begin{aligned} Y_1 &= (F_{y,fl} + F_{y,fr}) \cos \delta + (F_{x,fl} + F_{x,fr}) \sin \delta, \\ Y_2 &= F_{y,rl} + F_{y,rr}. \end{aligned}$$

This matches the model's own force aggregation.
`my_model`

4. Important identifiability conditions

The method will work on clean data only if these conditions are satisfied:

First, the car must actually generate lateral force. If Y_1 and Y_2 are near zero, the denominator of the estimator is near zero and d is not identifiable.

Second, do not estimate d , $k_{\phi 1}$, $k_{\phi 2}$, $c_{\phi 1}$, and $c_{\phi 2}$ all at once at the beginning. In the equation

$$t_i \Delta Z_i = Y_i q_i + k_{\phi i} \phi + c_{\phi i} \dot{\phi},$$

these terms can compensate each other. If k_{ϕ} and c_{ϕ} are uncertain, then the simple estimator identifies an effective d , not a pure geometric parameter.

Third, use transient steering, not only steady-state cornering. In steady state, $\dot{\phi} \approx 0$, and ϕ is strongly correlated with lateral acceleration. Transient sine/step/ramp tests give better separation between the geometric term $Y_i d$, the elastic term $k_{\phi} \phi$, and the damping term $c_{\phi} \dot{\phi}$.

Fourth, avoid tyre saturation for the first identifiability test. In saturated conditions, small changes in load can strongly change tyre force distribution, so the estimator still works algebraically if you use logged Y_i , but interpreting the result becomes harder.

5. Best simulation campaign for clean-data identification

For the first test, I would use your current phased steering setup: accelerate to target speed, hold, release drive torque, wait for the steering delay, then steer. Your code already defines the torque-release time and steering-start time, and for the phased steering profiles the steering starts after `t_steer_start = t2 + t_steer_delay`.

Use these cases:

Test	Speed	Input	Purpose
Smooth step steer	40, 60, 80 km/h	2°, 5°, 8°	Checks direct transient load transfer
Sine steer	40, 60, 80 km/h	0.3, 0.6, 1.0, 1.5 Hz	Separates $Y_i d$, $k_{\phi} \phi$, $c_{\phi} \dot{\phi}$
Slow ramp steer	40, 60 km/h	low ramp rate	Quasi-steady sanity check
Left/right steering	same amplitudes	positive and negative	Checks sign convention and bias

I would start with `ode15s` or `ode15i` and use the smoothed step input. The current code has a half-cosine smoothed step pulse specifically to avoid giving `ode15i` an ideal discontinuity.

6. The actual validation plan

Do not jump directly to noisy identification. I would do this sequence:

Stage 1 — algebraic recovery test

Run one clean simulation with the current true value:

$$d_{true} = 0.7h.$$

Then apply the estimator above using the logged simulation data.

Expected result:

$$\hat{d} \approx 0.7h$$

to numerical tolerance.

If it does not recover `veh.d` from clean data, then one of these is wrong: indexing of outputs, sign convention for ΔZ_i , reconstruction of Y_1 , Y_2 , or use of actual steering angle.

Stage 2 — front/rear consistency test

Estimate q_1 and q_2 separately.

Expected result:

$$\hat{q}_1 \approx \hat{q}_2 \approx d.$$

If $\hat{q}_1$ and $\hat{q}_2$ are not close in clean data, then there is likely an implementation/indexing/sign issue.

Stage 3 — multi-value recovery test

Run the same simulation with several values:

$$d/h = 0.2, 0.4, 0.6, 0.8.$$

For each case, rebuild the model and rerun the simulation. Then check whether the estimator recovers the value used to generate the data.

Important implementation warning: in your current CasADi setup, `veh.d`, `veh.H`, `veh.q1`, and `veh.q2` are numerical constants when `my_model.m` is run and the CasADi functions are built. The script builds single-output CasADi functions from `my_model.m`.

So for a parameter sweep, simply changing `model.veh.d` after model construction is not enough. You need to either rebuild the model each time or refactor `d` as a symbolic parameter.

Stage 4 — noise robustness test

Add artificial noise to the clean logged signals:

$$F_{x_1}, F_{x_2}, F_{y_1}, \phi_1, \dot{\phi}_1, \delta.$$

Then repeat the LS estimation and plot error versus noise level.

This tells you whether real data has any chance.

Stage 5 — nonlinear output-error identification

Only after the algebraic test works should you use a nonlinear identification problem:

$$\min_d \sum_k \|y_{meas,k} - y_{sim,k}(d)\|_W^2, \quad 0 \leq d \leq h.$$

This uses the full dynamic effect of $H = h - d$ and the algebraic effect $q_1 = q_2 = d$. But it is much more expensive because each candidate `d` needs a simulation. For your current code, that means rebuilding the model for each candidate unless `d` is turned into a CasADi parameter.

7. Final judgement

My previous advice is valid, but it needs this sharper wording:

The direct LS method is rigorous and implementable for identifying q_1 , q_2 , and because your current code sets $q_1 = q_2 = d$, it can back-identify `veh.d` from clean simulation data.

But:

It is not enough to claim that real vehicle-motion data alone can identify `veh.d`. For that, you need either measured wheel loads/tyre forces or a carefully designed nonlinear output-error identification with sensitivity/rank checks.

For your immediate purpose, the clean-simulation experiment is exactly the right first step. If the direct estimator cannot recover `veh.d` from your own generated clean data, then real-data identification is not worth attempting yet.

📄 🔍 ↺ ⬆ ⬇ ⬅ ⋮ 📄 資料來源

到目前為止，這段對話有幫助嗎？ 🔍 ↺ ✕